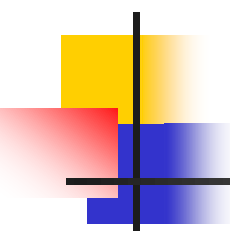


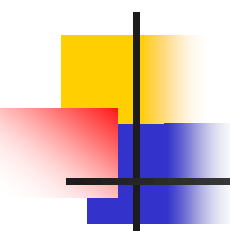
Introduction to Neural Networks and its Applications

Prof. Nadir Charniya, (Ph.D Electronics Engg.)



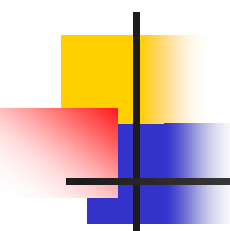
Outline

- Introduction to Neural Networks
- Artificial Neuron
- Perceptrons
- Multilayer Perceptron
- Backpropagation training algorithm



So What's *Artificial* Neural Networks

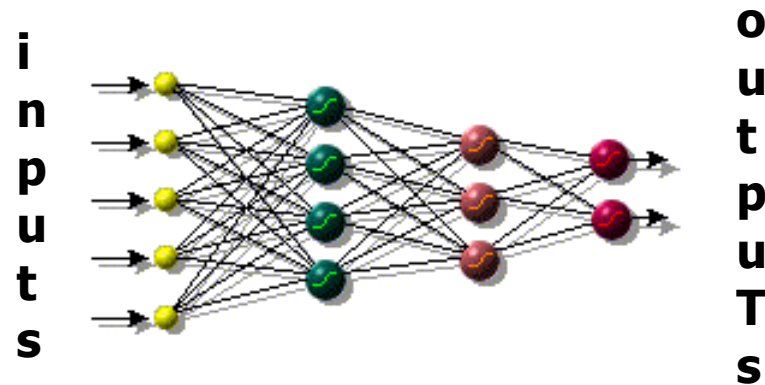
- An *artificial* neural network is mans crude way of trying to simulate (using H/W or S/W) the brain electronically
- Def. : *An Artificial Neural Network (ANN) is a massively connected structure of Artificial Neurons that has a natural tendency for strong experiential knowledge and making it available for use.*
- *It resembles the brain in two respects,*
 - Knowledge Acquisition through learning
 - Knowledge is stored in synaptic weights



Artificial Neural Networks

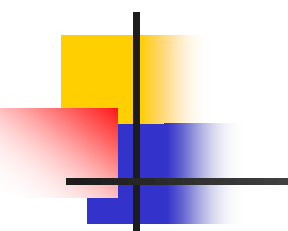
- **Powerful mathematical algorithms, which mimic the human brain's ability to learn**
- **Used when first principle equations of system cannot easily be developed**
- **Used for Regression, Clustering, and Classification**

Human Brain and Artificial Neural Networks



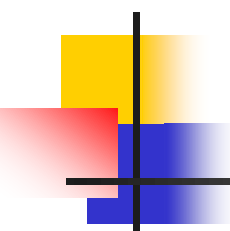
Brain has different centers for input recognition
and control

e.g Soft/Hard



Supervised and unsupervised Learning

- **Supervised:** (target o/p is known)
(e.g Supervised learning in new born child: Mothers touch, voice; Twins)
- **Unsupervised** (No teacher or reference; network learns intrinsically from data)
e.g Unsupervised clustering in Marriage Reception Hall

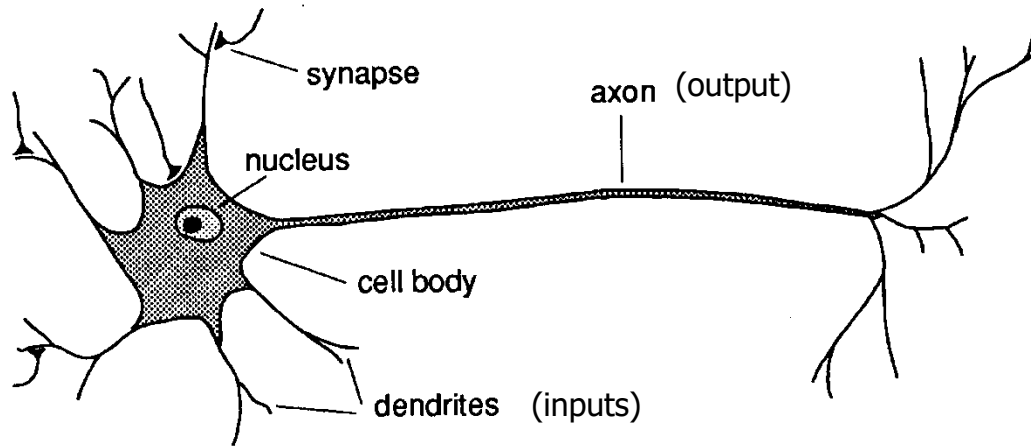


Need of Neural Networks

The special features of neural networks

- **Capability to learn from examples**
- **Ability to generalize results obtained from known situations to unforeseen situations**
(colour shade recognition by child 'Black', 'Gray', and 'White')
- **Intrinsically capable of recognizing a complex pattern**
- **Robustness to noise**
(Recognition on image corrupted by noise)
- **Fast response due to a high degree of structural parallelism**

Schematic Drawing of a Biological Neuron



- dendrites - inputs
- axon - output
- synaptic junctions - excitatory or inhibitory
- activation potential, threshold and firing of cell
- Collecting different stimuli from dendrites, and when the accumulative input (summation) reaches a certain level , the axon (output) , *fires* , and transmits the data to the next node (e.g sense of heat and reaction by body)

It's all about Electro-chemical Activity

Where is Knowledge Stored ?

Receives Electrochemical Activity then fires when a threshold is exceeded

Dendrites

Soma

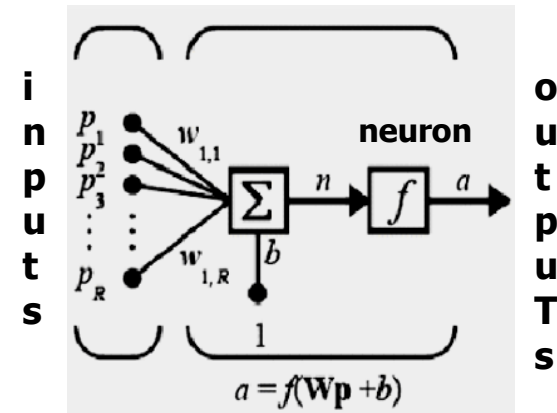
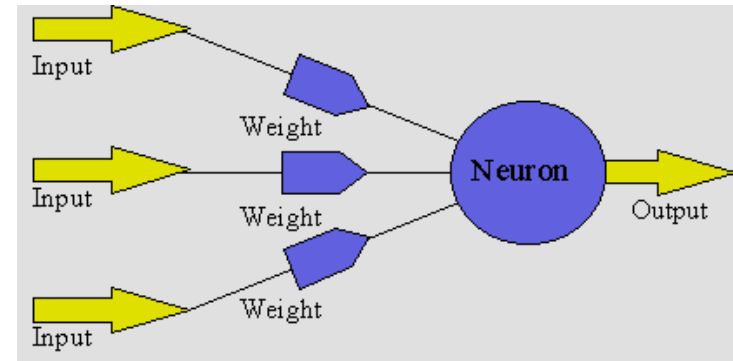
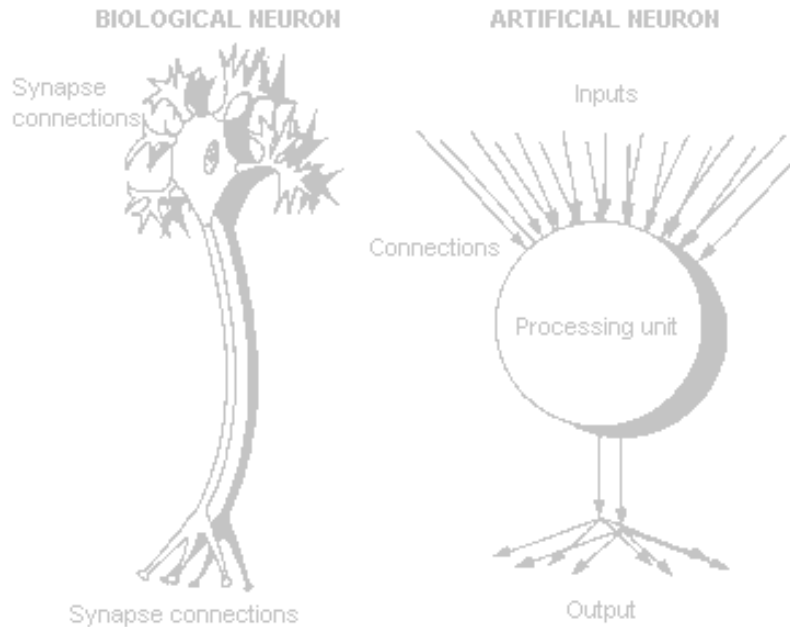
Axon

*Knowledge is stored in the chemical medium in the Synaptic Junction
(Synaptic Knowledge Weights)*

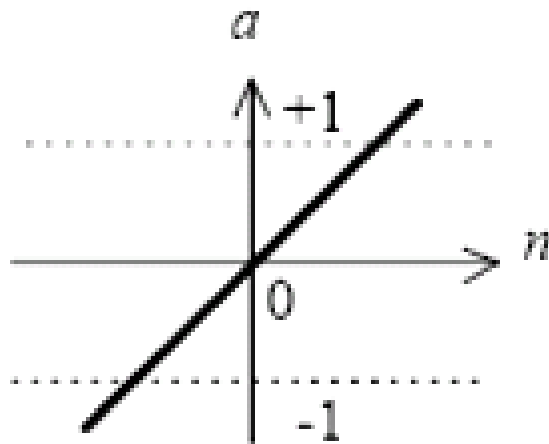
Synaptic Junction

2/11/2022

Inspiration from Brain Neurons



Activation Functions

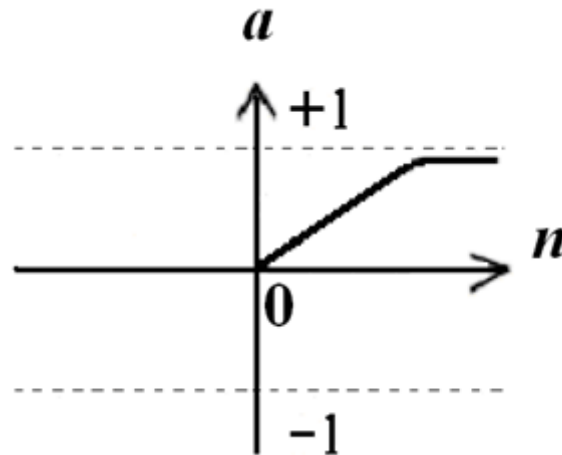


$$a = \text{purelin}(n)$$

$$a = n$$

Linear transfer function

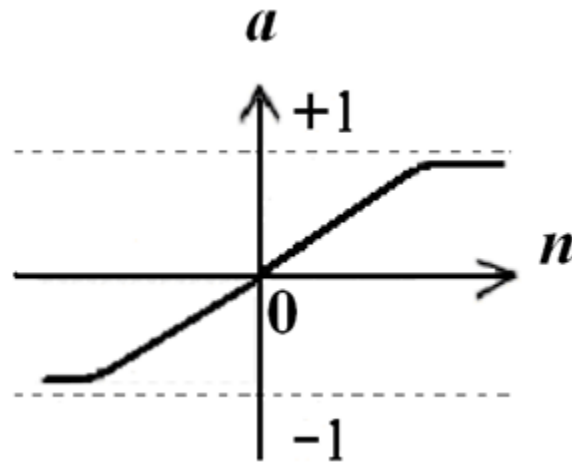
Linear unipolar with saturation



$$a = \begin{cases} 1 & n > 1 \\ n & \text{otherwise} \\ 0 & n < 0 \end{cases}$$

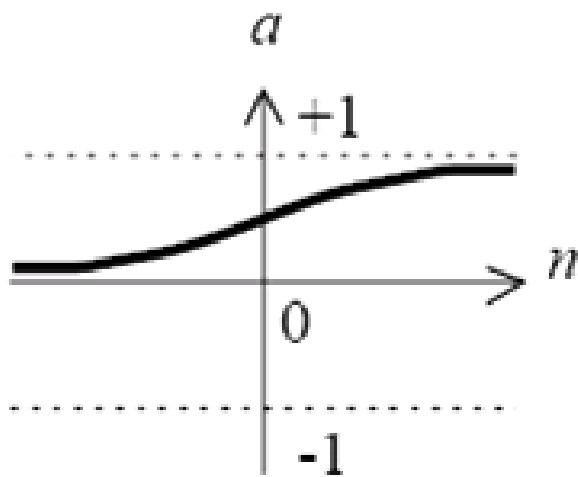
n is net

Linear bipolar with saturation



$$a = \begin{cases} 1 & n > 1 \\ n & \text{otherwise} \\ -1 & n < -1 \end{cases}$$

n is net



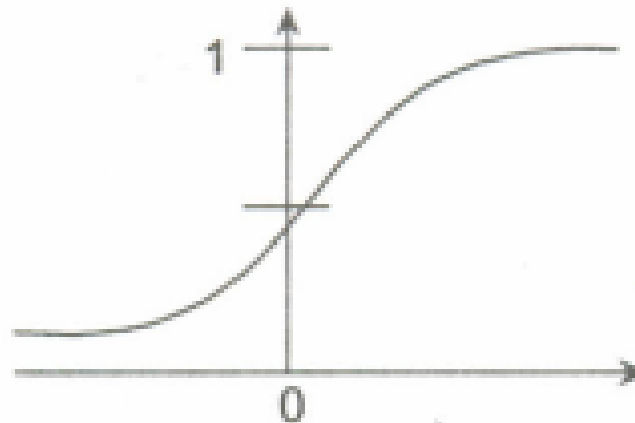
$$a = \text{logsig}(n)$$

$$a = \frac{1}{1 + e^{-n}}$$

Sigmoid transfer function

Sigmoidal function

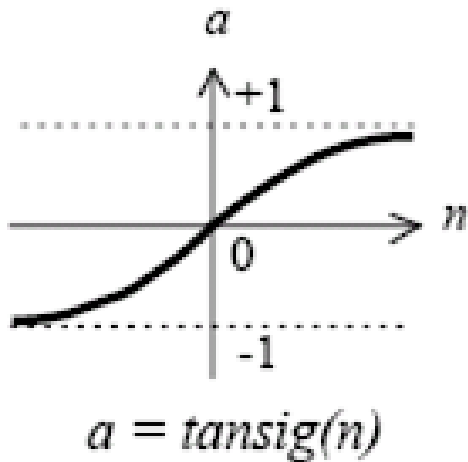
CONTD



$$y = \frac{1}{(1 + \exp(-\lambda l))}$$

here l is net value

Bipolar sigmoid (Tanh)

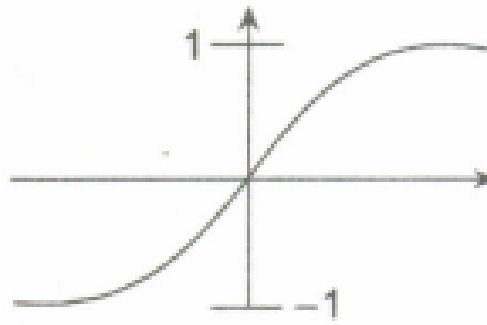


$$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$$

Hyperbolic tangent function

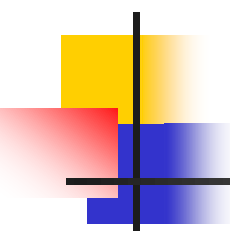
Bipolar Sigmoid

CONTD



$$y = \tanh. [\lambda I]$$

here I is net value



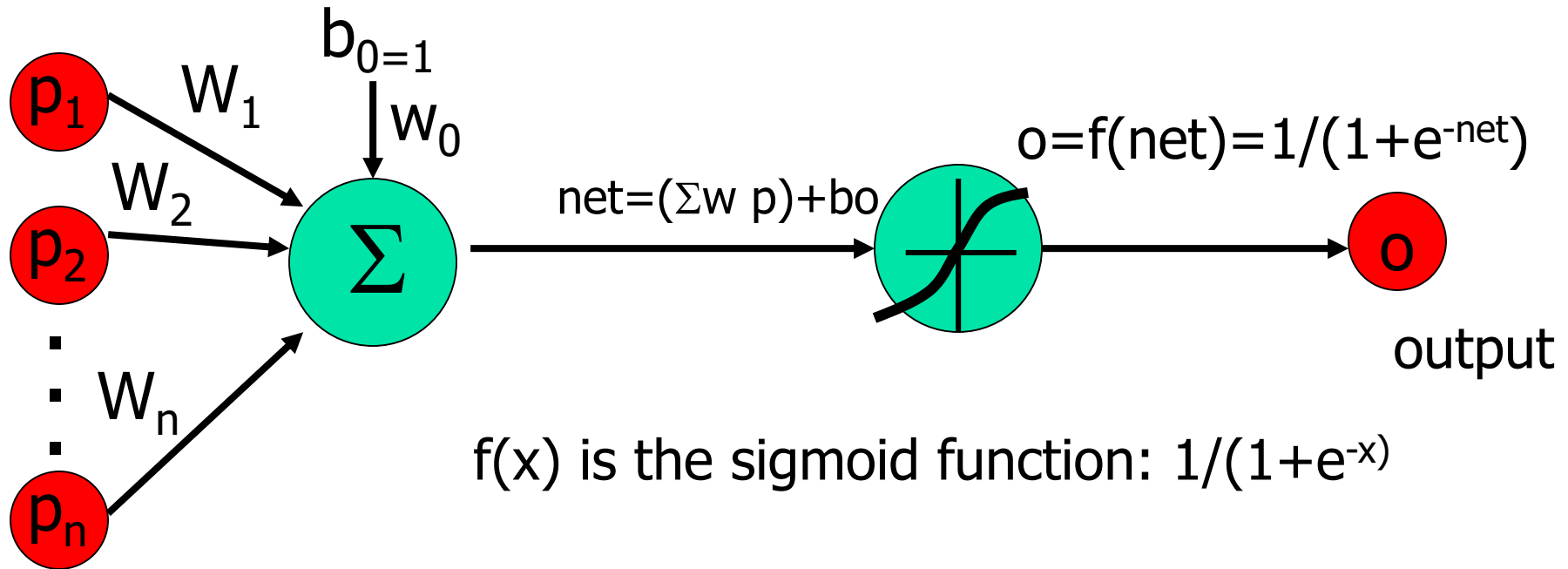
Threshold Activation Function

Threshold θ or T

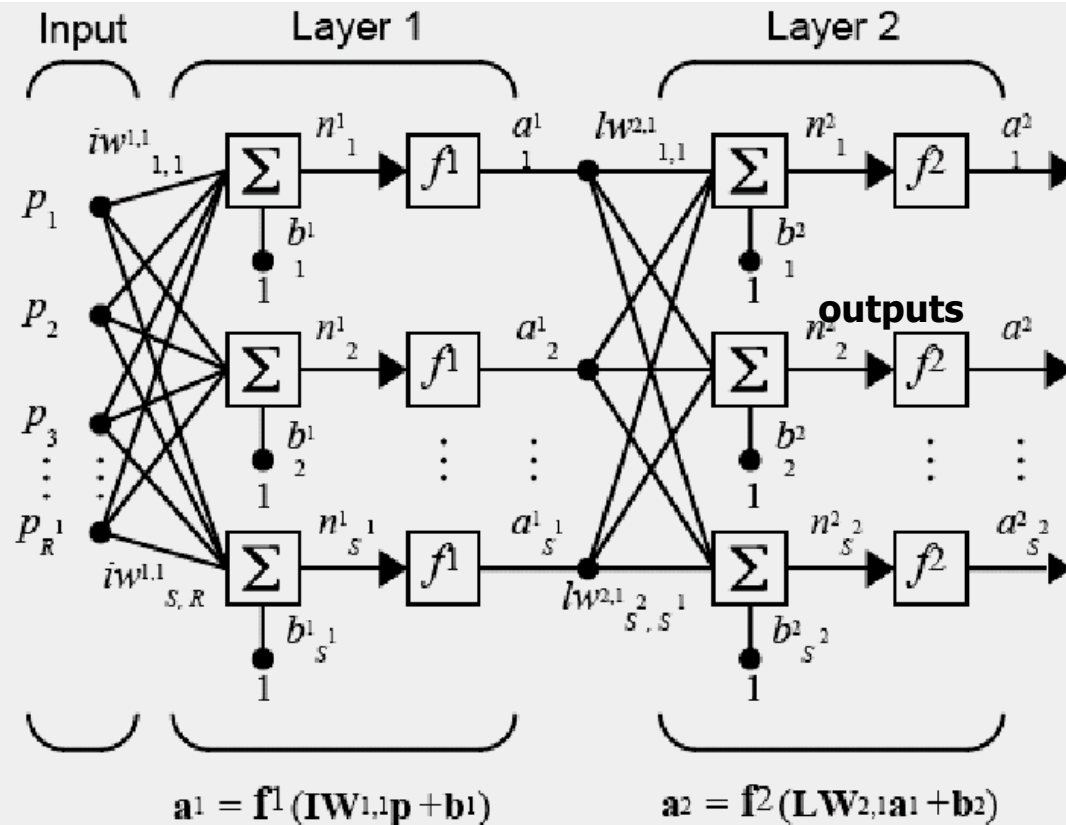
$$(i) \quad y = f(\text{Net}) = \begin{cases} +1 & \text{if } \text{net} \geq \theta; \\ -1 & \text{if } \text{net} < \theta; \end{cases}$$

$$(ii) \quad y_j = f(\text{Net}) = \begin{cases} 1 & \text{if } y_{inj} > \theta_j \\ y_j & \text{if } y_{inj} = \theta_j \\ -1 & \text{if } y_{inj} < \theta_j \end{cases}$$

Sigmoid Neuron



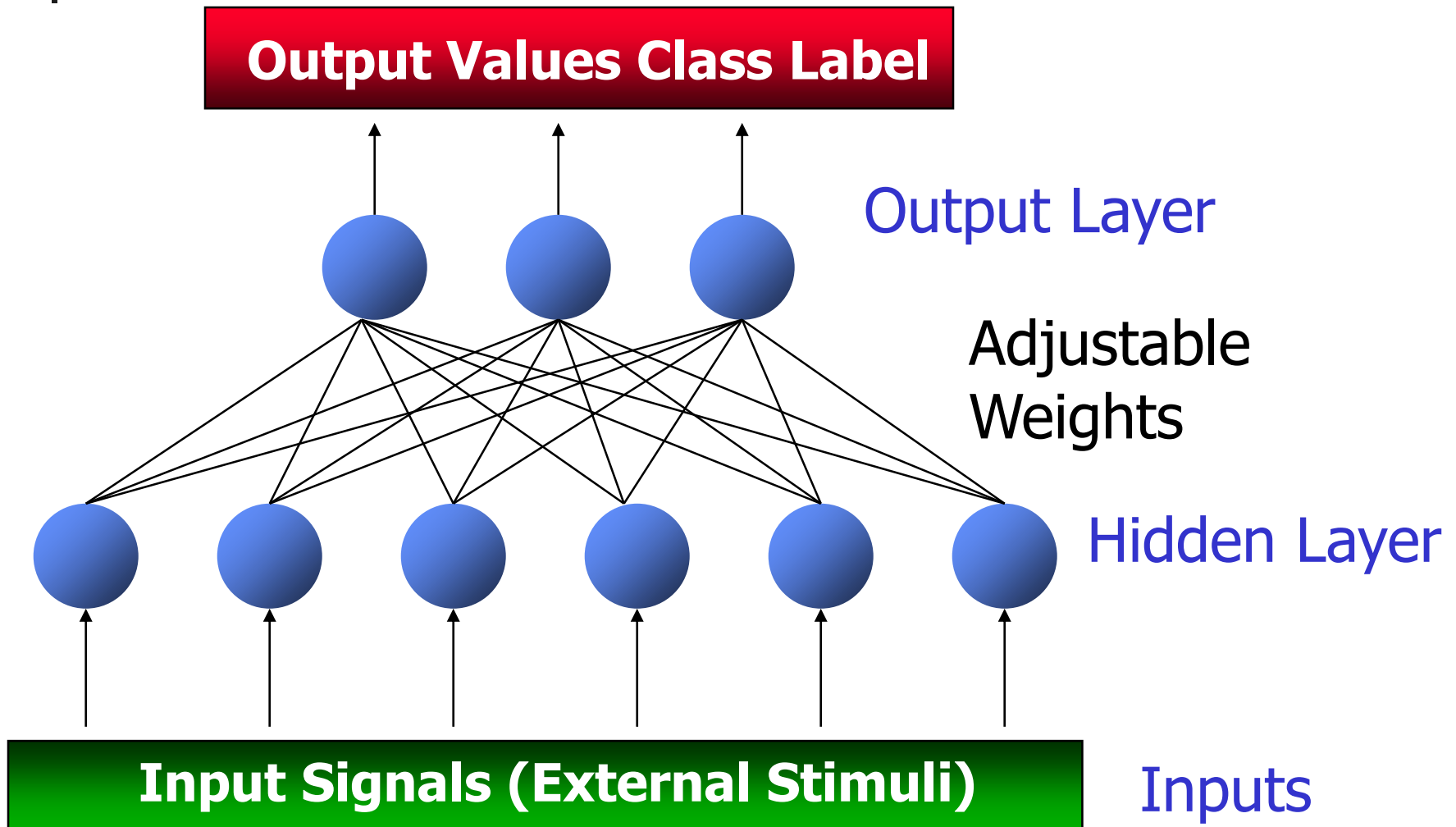
Multiple Neurons connections in Multilayer Perceptron Structure

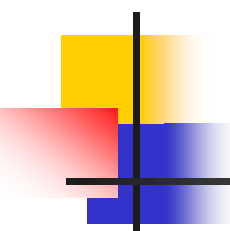


$$a^2 = f^2(W^2 f^1(W^1 p + b^1) + b^2)$$

Fig. 2.5 a) A two-layer MLP NN

Multilayer Perceptron (MLP)

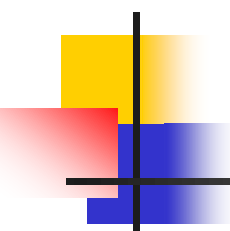




Data Collection

The data-collection phase must be carefully planned to ensure that

- ❑ **Data will be sufficient**
- ❑ **Data will capture the fundamental principles at work**
- ❑ **Data is as free as possible from observation noise.**



Partition of Data Sets

- **Training Data Set:**

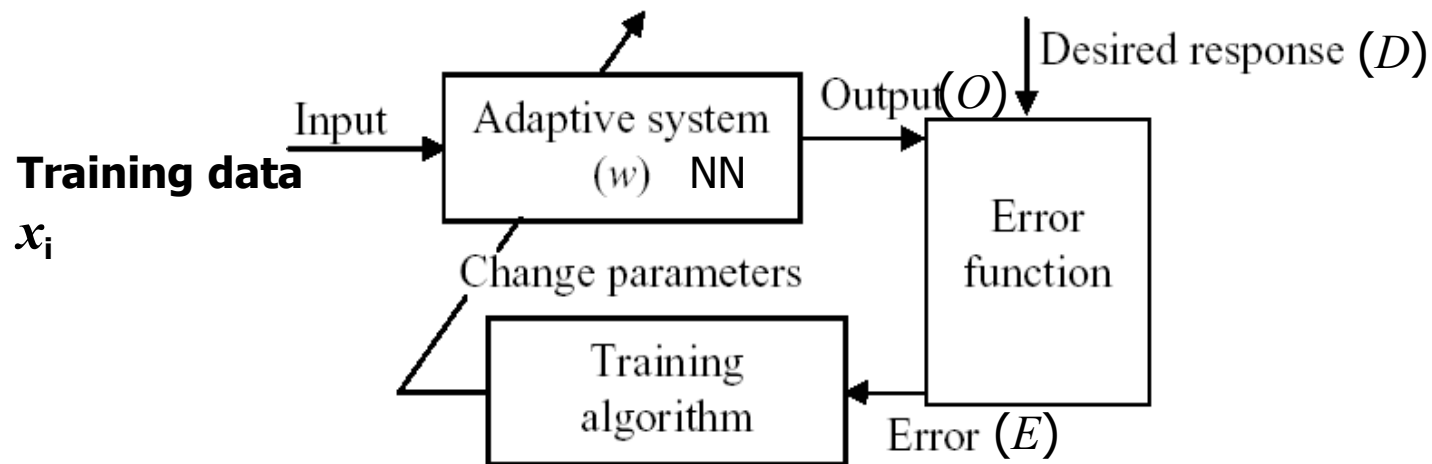
Data used for determination of optimal network weights by training the Neural Network

- **Testing Data Set:**

Data used to test the performance of the trained neural network

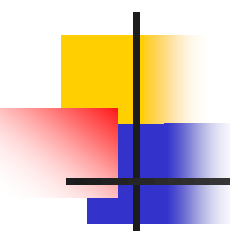
How to determination of Weights:

*Training the supervised **Feed forward** Neural Networks using **Back propagation** Learning Algorithm*



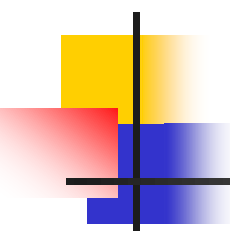
Mean Square Error:
$$E = \sqrt{\sum_j (D_j - O_j)^2}$$

Training:
$$w = w + \Delta w$$



Back-Propagation

- A training procedure which allows multi-layer feedforward Neural Networks to be trained;
- Can theoretically perform “any” input-output mapping;
- Can learn to solve linearly inseparable problems using MLP NN.



Back Propagation Learning

i.e Widrow Hoff / LMS

Set initial NW weights as random small values

$$w_i = w_i + \Delta w_i$$

$$\Delta w_i = \eta (D - O) x_i$$

$$w(k+1) = w(k) + \eta \varepsilon(k) x(k)$$

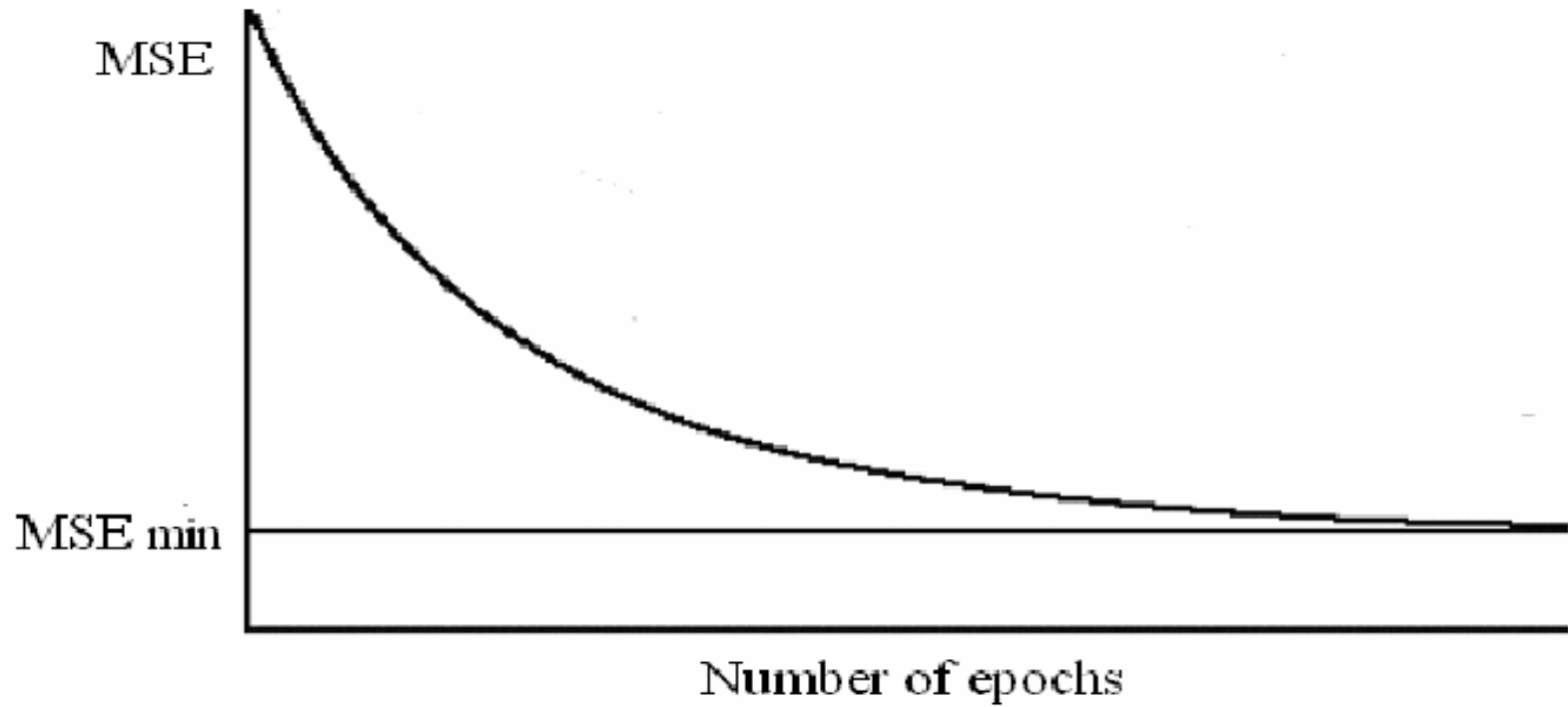
D is the desired target value

O is the network actual output

η Is a small constant (e.g. 0.1) called *learning rate*

- If the output is correct ($D \equiv O$) i.e $E \equiv 0$ the weights w_i are not changed
- If the output is incorrect ($D \neq O$) the weights w_i are changed such that the output of the network for the new weights is *closer* to D .

Learning curve



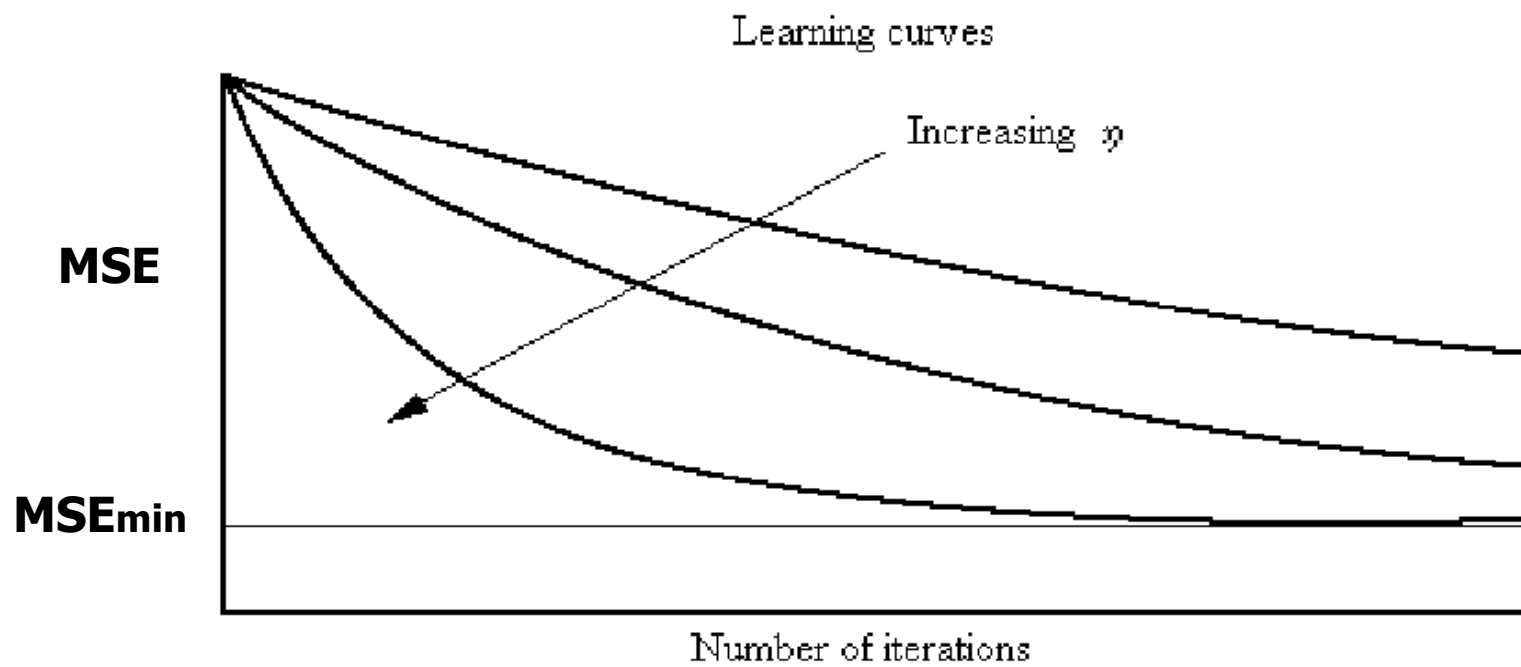
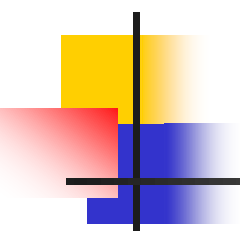
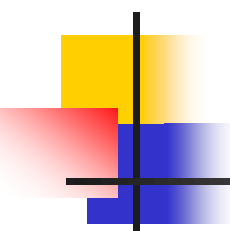


Figure 1-10 The learning curve



Learning algorithm

Epoch : Presentation of the entire training set to the neural network.

In the case of the **AND function** an epoch consists of four sets of inputs being presented to the network (i.e. **[0,0], [0,1], [1,0], [1,1]**)

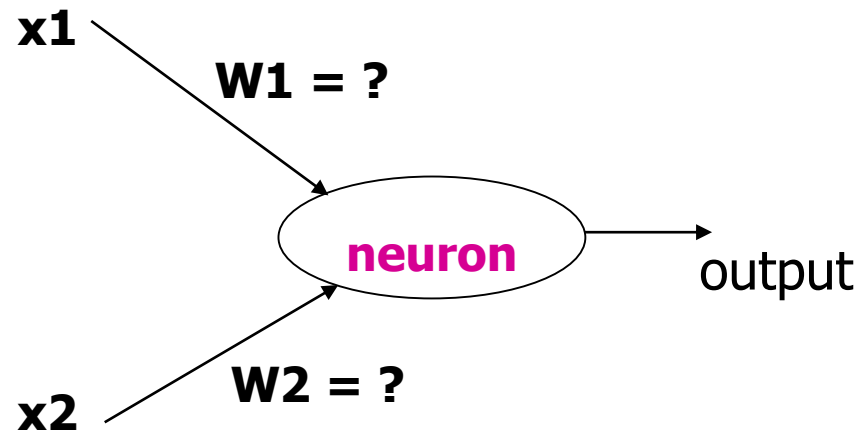
Error: The error value is the amount by which the actual output value by the network differs from the desired target value.

AND		
x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Example:

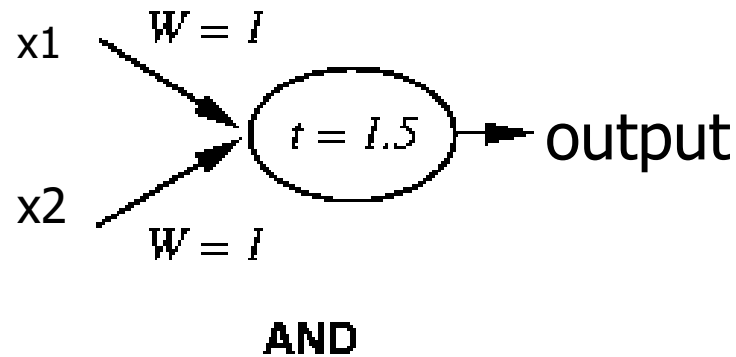
Training the NN (Weights=?) for AND logic

AND		
x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1



- What are the weight values?
- Initialize with small random weight values

Simple Network of AND

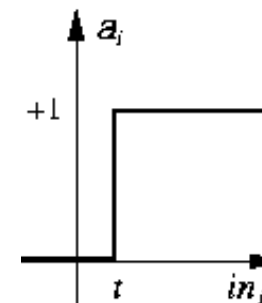


AND

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

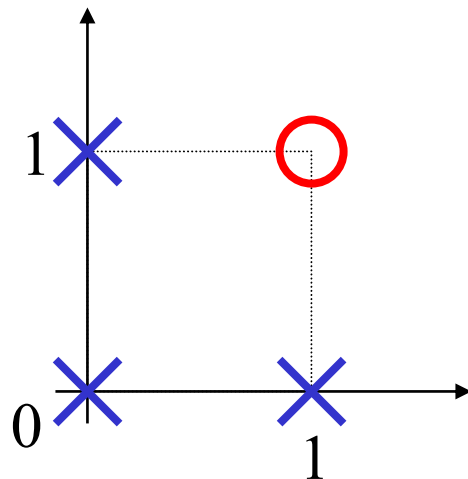
$$\text{Output} = \begin{cases} 1 & \text{if } \sum_{i=0} w_i x_i > t \\ 0 & \text{otherwise} \end{cases}$$

- Linear threshold is used.
- W - weight value
- t - threshold value



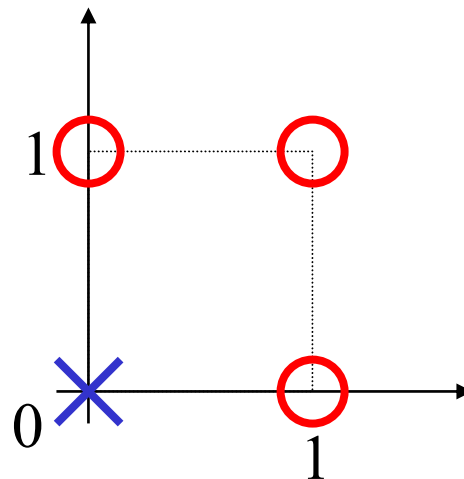
Linearly Separable vs. Linearly Non-Separable

AND



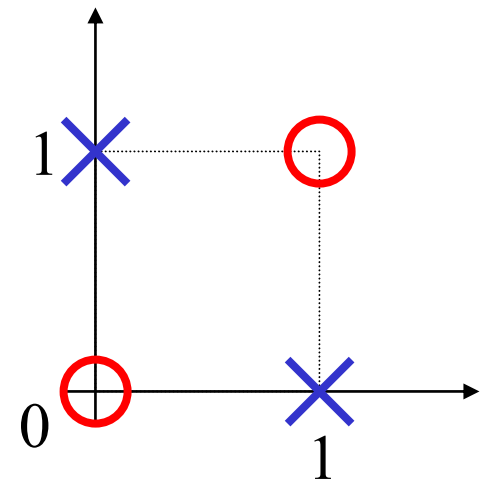
Linearly Separable

OR



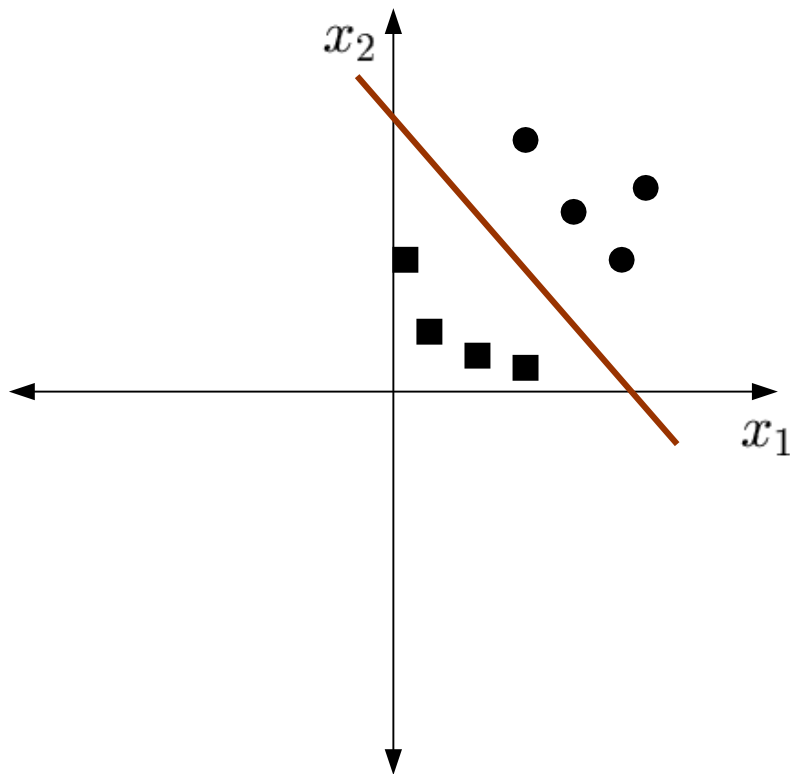
Linearly Separable

XOR

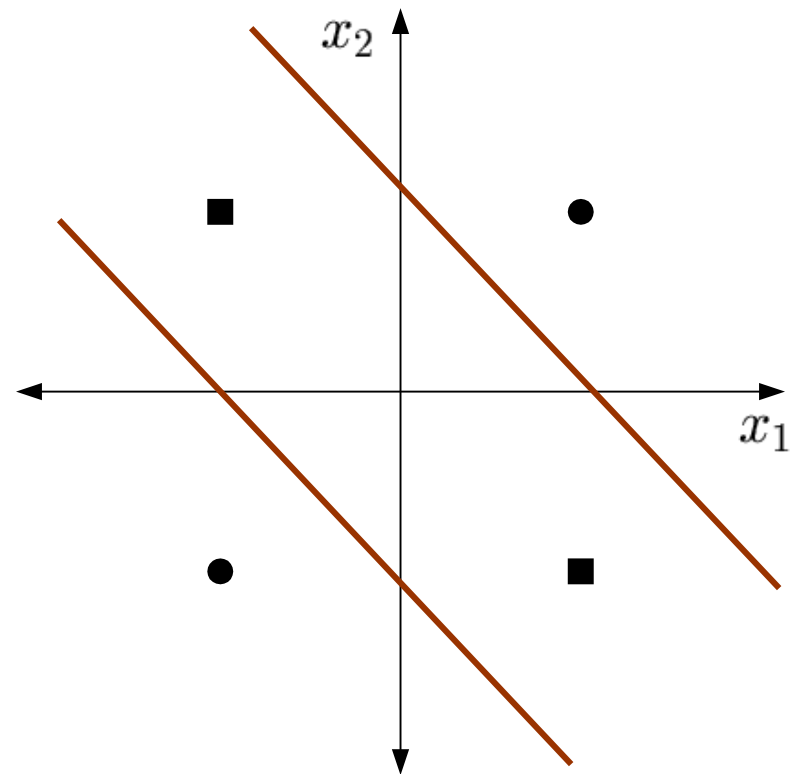


Linearly Non-Separable

Decision Boundary:: x_1 and x_2 are inputs

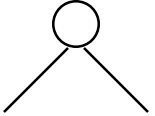
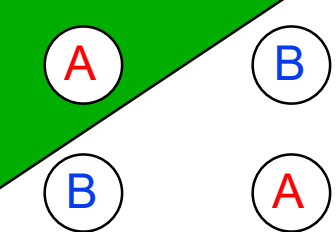
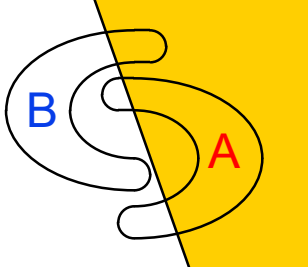
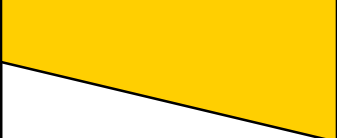
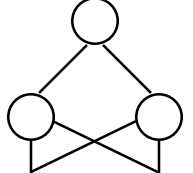
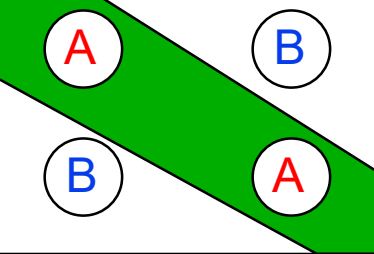
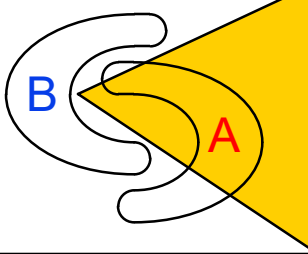
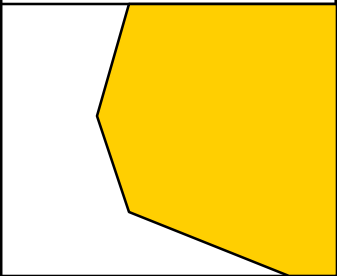
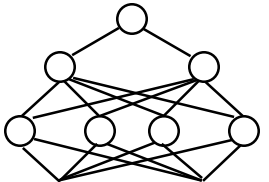
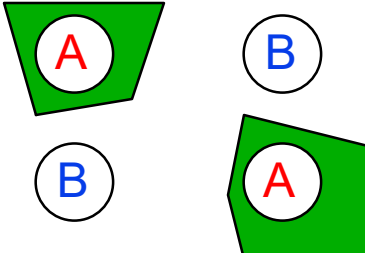
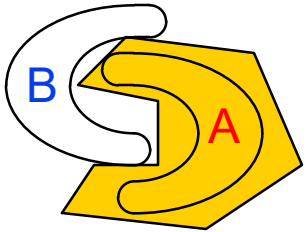
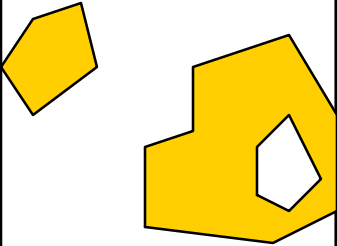


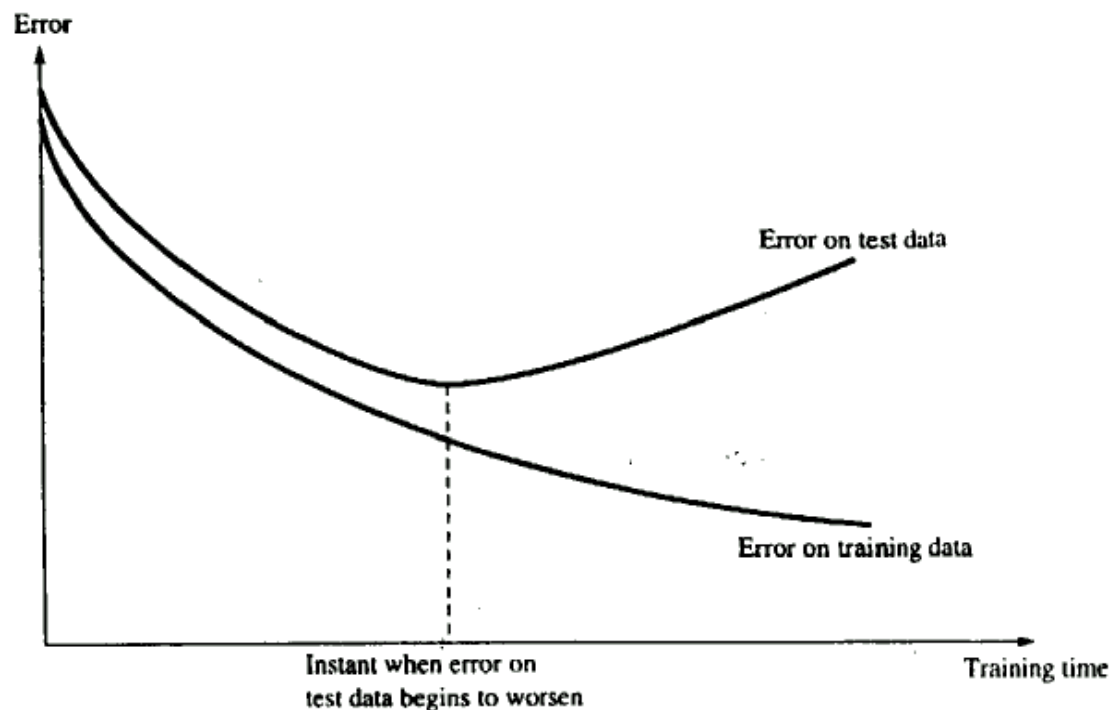
Linearly separable e.g AND,
OR

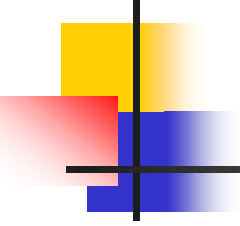


Nonlinearly separable e.g EXOR

Different Non-Linearly Separable Problems

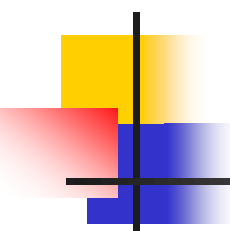
Structure	Types of Decision Regions	Exclusive-OR Problem	Classes with Meshed regions	Most General Region Shapes
Single-Layer 	<i>Half Plane Bounded By Hyperplane</i>			
Two-Layer 	<i>Convex Open Or Closed Regions</i>			
Three-Layer 	<i>Arbitrary (Complexity Limited by No. of Nodes)</i>			





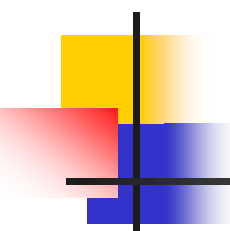
Training time

- How many epochs of training?
 - Stop if the error fails to improve (has reached a minimum)
 - Stop if the rate of improvement drops below a certain level
 - Stop if the error reaches an acceptable level
 - Stop when a certain number of epochs have passed



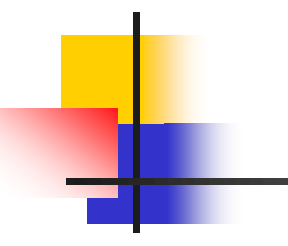
Optimal NN Parameter setting

- Number of layers, Number of neurons
 - too many neurons, require more training time
 - overfitting (Memorization); No Generalization
- Learning rate
 - From experience, small value ~ 0.1
- Neuron transfer function
 - from number of trials
- Number of training epochs



Over-fitting

- Over training and more nodes lead to over fitting or memorization of training set
- May have poor generalisation ability.
- Cross-validation with some patterns
 - Typically 30% of training patterns
 - Validation set error is checked each epoch
 - Stop training if validation error goes up



Gradient Descent, Delta Rule (Gradient) Steepest descent, S

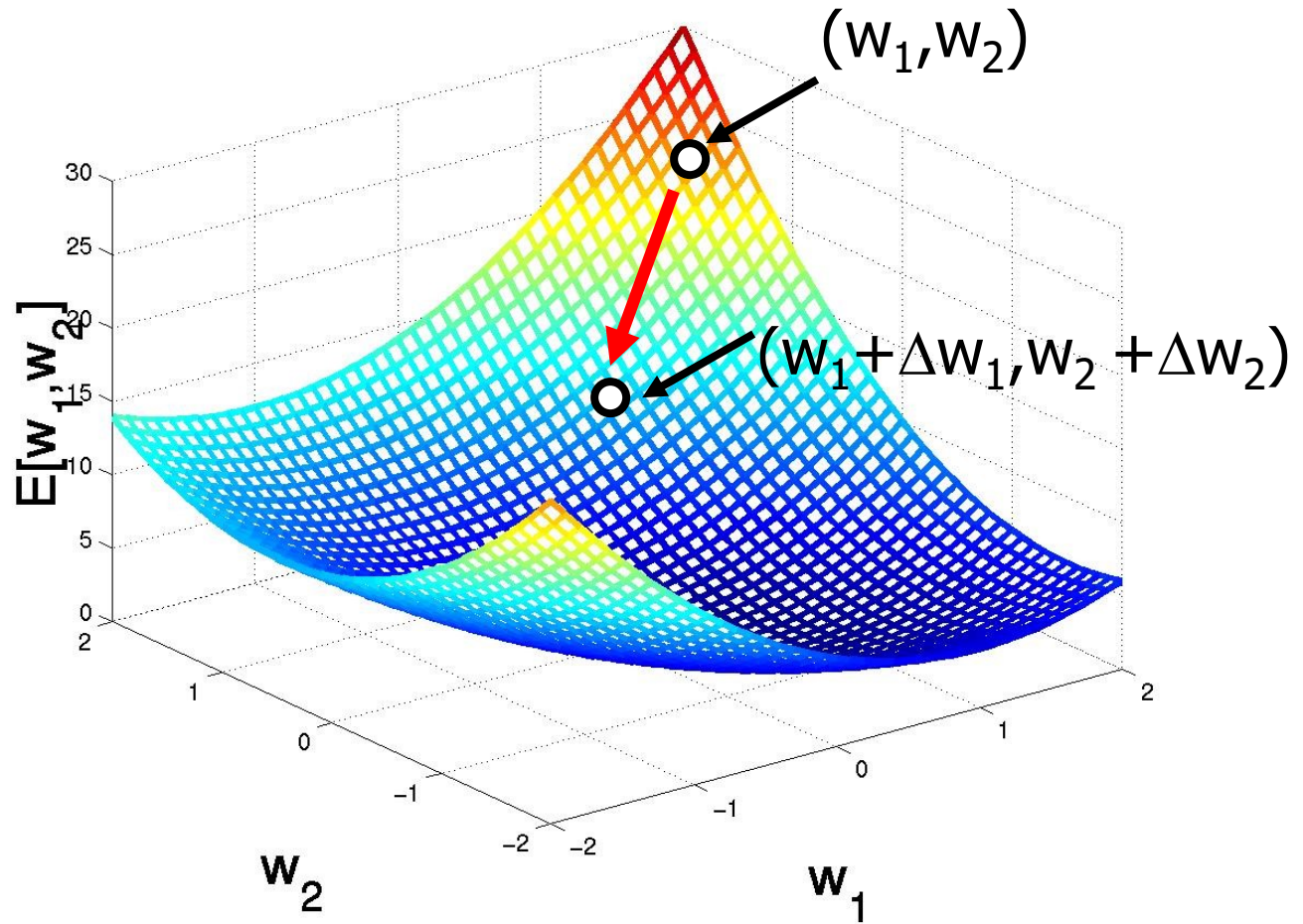
The delta training rule is best understood by considering the task of training an *unthresholded* perceptron; that is, a *linear unit* for which the output o is given by

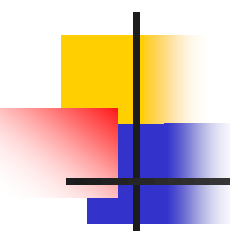
$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

In order to derive a weight learning rule for linear units, let us begin by specifying a measure for the training error of a hypothesis (weight vector), relative to the training examples.

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Visualizing the Hypothesis Space





Derivation of the Gradient Descent Rule

The vector derivative is called the *gradient* of E with respect to $\vec{w}$,
 $\nabla E(\vec{w})$ written

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

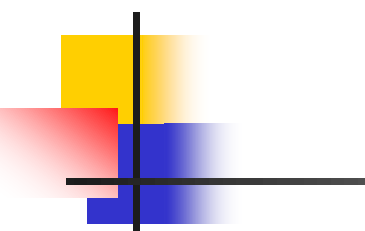
The gradient specifies the direction that produces the steepest increase in E . The negative of this vector therefore gives the direction of steepest decrease. The training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w}$$

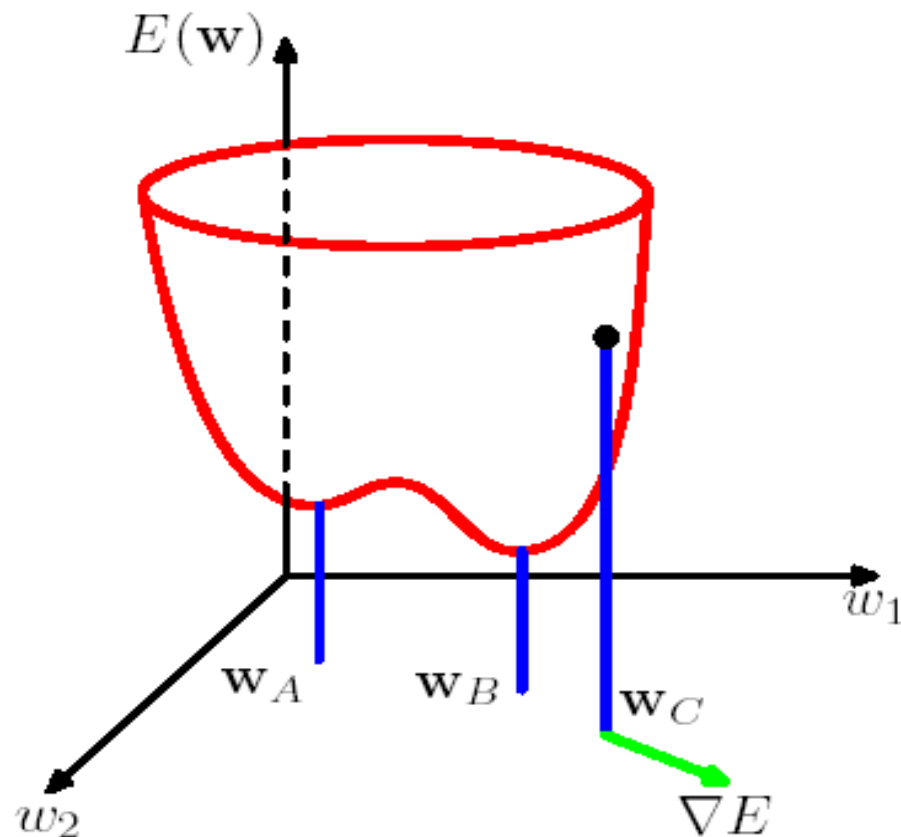
where

$$\Delta w_i = \textit{Alpha} (t - \textit{o}) x_i$$

$$\Delta \vec{w} = -\eta \nabla E(\vec{w})$$



Geometrical view of the error function $E(\mathbf{w})$ as a surface sitting over weight space. Point $\mathbf{w}_A$ is a local minimum and $\mathbf{w}_B$ is the global minimum. At any point $\mathbf{w}_C$, the local gradient of the error surface is given by the vector ∇E .





Derivation of the Gradient Descent Rule (cont.)

The negative sign is presented because we want to move the weight vector in the direction that *decreases* E . This training rule can also be written in its component form

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

which makes it clear that steepest descent is achieved by altering each component w_i of $\vec{w}$ in proportion to $\frac{\partial E}{\partial w_i}$.

Derivation of the Gradient Descent Rule (cont.)

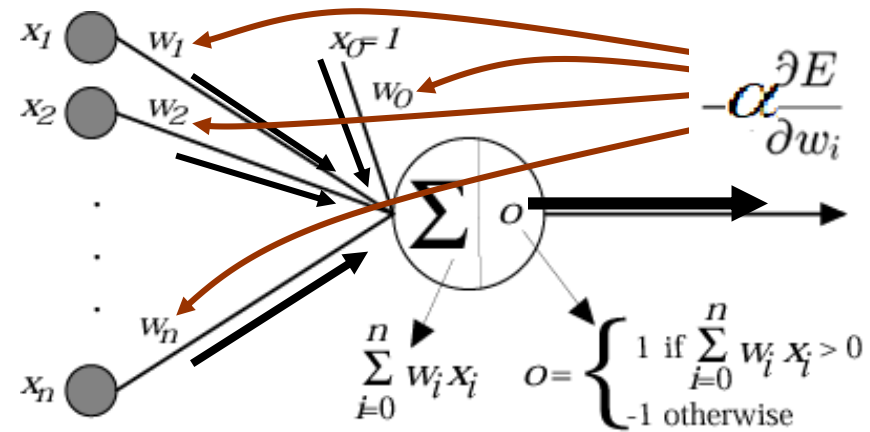
The vector of $\frac{\partial E}{\partial w_i}$ derivatives that form the gradient can be obtained by differentiating E

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\ \frac{\partial E}{\partial w_i} &= \sum_{d \in D} (t_d - o_d) (-x_{id})\end{aligned}$$

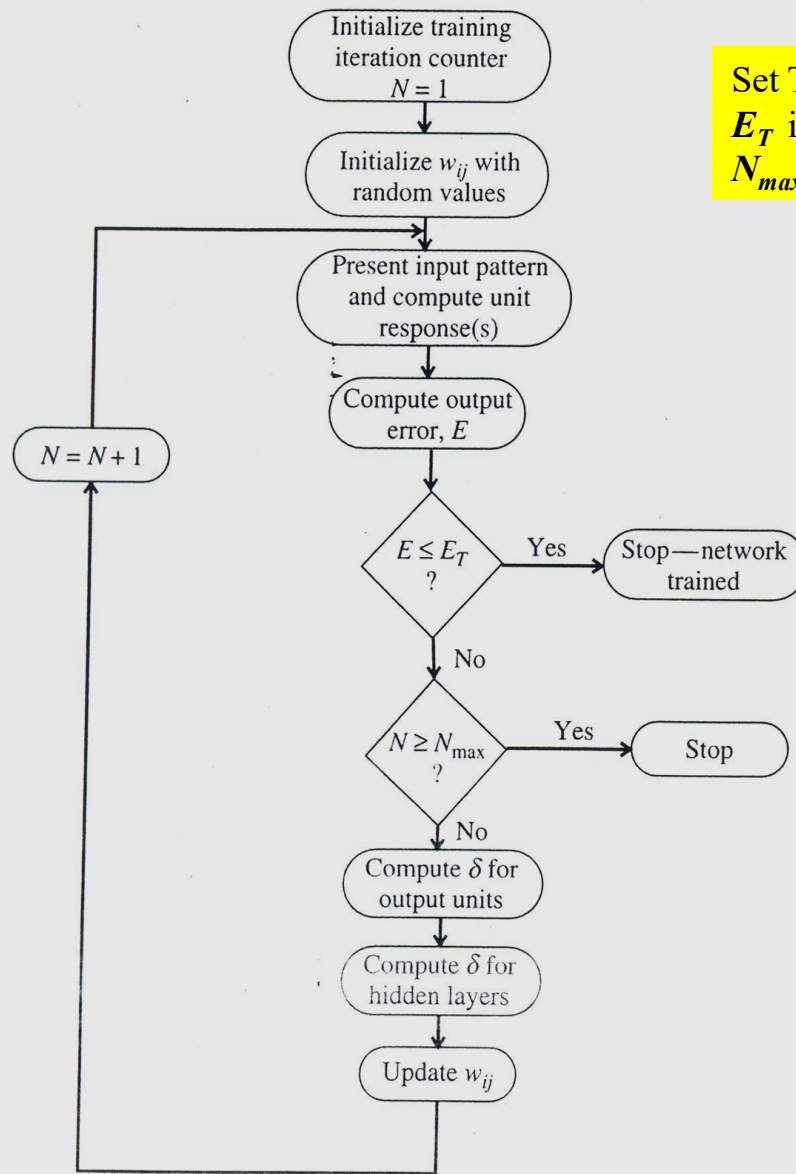
The weight update rule *for standard gradient descent* can be summarized as

$$\begin{aligned}\Delta w_i &= -\eta \frac{\partial E}{\partial w_i} \\ \Delta w_i &= \eta \sum_{d \in D} (t_d - o_d) x_{id}\end{aligned}$$

Stochastic Approximation to Gradient Descent

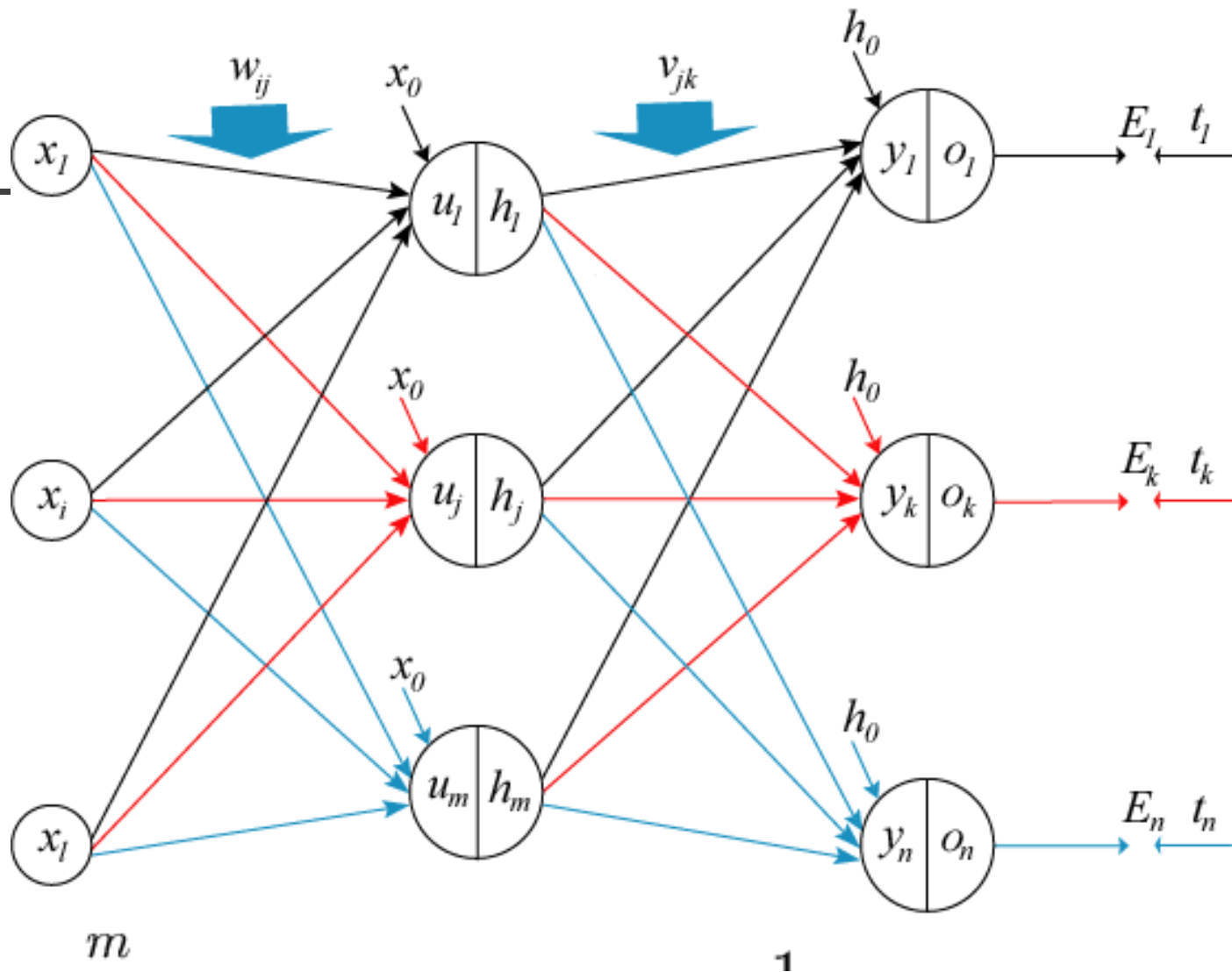


Generalized Delta Rule (Gradient Descent)



Set Thresholds for
 E_T i.e. Minimum error;
 N_{max} i.e. Max Epochs

Architecture of MLP NN



$$w_{ij}^{(new)} = w_{ij}^{(old)} + \eta \cdot \delta h_j \cdot x_i, 0 \leq i \leq l, 1 \leq j \leq m$$



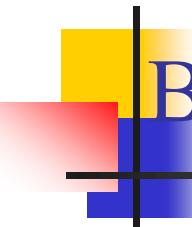
Backpropagation Learning Algorithm

Step 1. Initialize the weights of the perceptron randomly with number between -0.1 and 0.1.

$$w_{ij} = \text{random}([-0.1, 0.1]) \quad 0 \leq i \leq l, 1 \leq j \leq m$$

$$v_{jk} = \text{random}([-0.1, 0.1]) \quad 0 \leq j \leq m, 1 \leq k \leq n$$

Step 2. Present the pattern $p = (x_1, x_2, x_i, \dots, x_l) \in \mathbb{R}^l$ to the perceptron, $p \in P$



Backpropagation Learning Algorithm (cont.)

Step 3. Compute the values of the hidden-layer nodes using the formula

$$u_j = \sum_{i=0}^l w_{ij} \cdot x_i, \quad h_j = \frac{1}{1 + e^{-u_j}}, 1 \leq j \leq m$$

Step 4. Calculate the values of the output nodes using the formula

$$y_k = \sum_{j=0}^m v_{jk} \cdot h_j, \quad o_k = \frac{1}{1 + e^{-y_k}}, 1 \leq k \leq n$$



Backpropagation Learning Algorithm (cont.)

Step 5. Compute the errors $\delta o_k, 1 \leq k \leq n$, in the output layer using

$$\delta o_k = (t_k - o_k)o_k(1 - o_k), 1 \leq k \leq n$$

Step 6. Compute the errors $\delta h_j, 1 \leq j \leq m$, in the hidden layer using

$$\delta h_j = h_j(1 - h_j) \sum_{k=1}^n \delta o_k \cdot v_{jk}, 1 \leq j \leq m$$



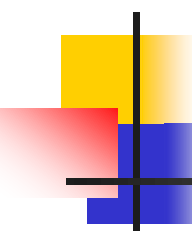
Backpropagation Learning Algorithm (cont.)

Step 7. Adjust the weights between the output layer and the hidden layer according to the formula

$$v_{jk}^{(new)} = v_{jk}^{(old)} + \eta \cdot \delta o_k \cdot h_j, 0 \leq j \leq m, 1 \leq k \leq n$$

Step 8. Adjust the weights between the hidden layer and the input layer according to

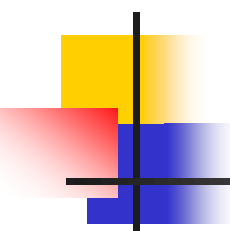
$$w_{ij}^{(new)} = w_{ij}^{(old)} + \eta \cdot \delta h_j \cdot x_i, 0 \leq i \leq l, 1 \leq j \leq m$$



Backpropagation Learning Algorithm (cont.)

Step 9. Repeat step 2 through 8 for each element of training set. Until $TSSE \leq \epsilon$ or converge the maximum iteration, ϵ is a error tolerance, where

$$TSSE = \frac{1}{2} \sum_{p \in P} \sum_{k=1}^n (t_k - o_k)^2,$$



Error Function

The Backpropagation algorithm learns the weights for a multilayer network, given a network with a fixed set of units and interconnections. It employs gradient descent to attempt to minimize the squared error between the network output values and the target values for those outputs. We begin by redefining E to sum the errors over all of the network output units

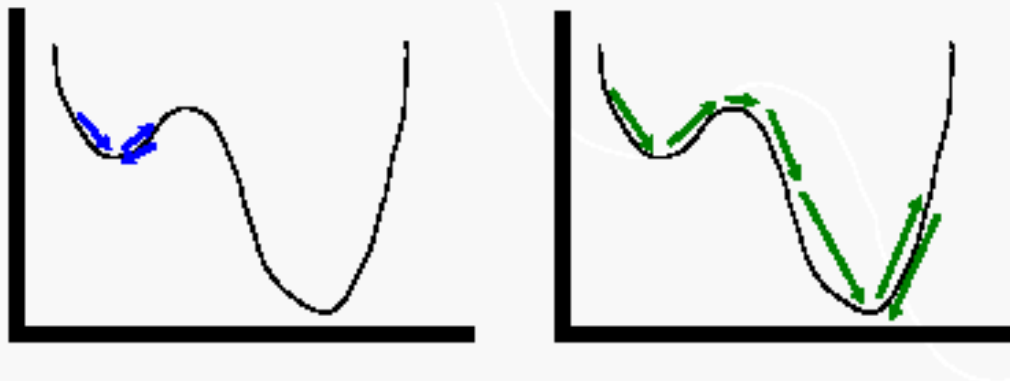
$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2$$

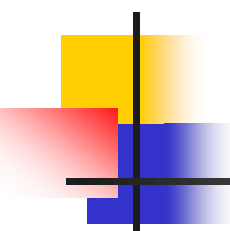
where *outputs* is the set of output units in the network, and t_{kd} and o_{kd} are the target and output values associated with the k^{th} output unit and training example d .

Variations of BP nets

Enhancements To Gradient Descent

- Momentum
 - Adds a percentage of the last movement to the current movement



- 
-
- It is possible to use a larger learning rate and avoid long flat error surfaces, while maintaining the stability of the algorithm
 - Often finds Minima in less steps
 - Error is no longer monotonically decreasing

Advantage of adding momentum

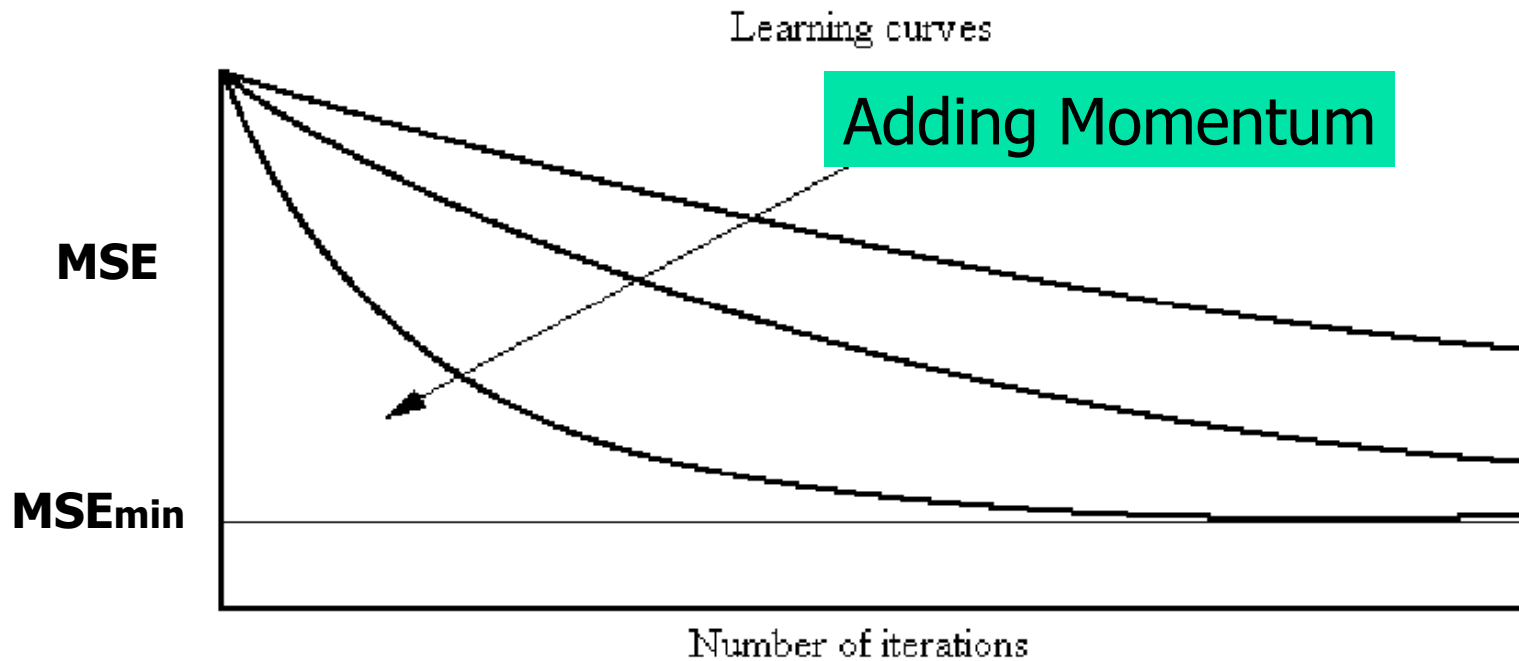
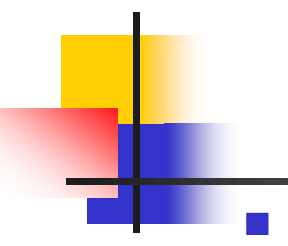


Figure 1-10 The learning curve



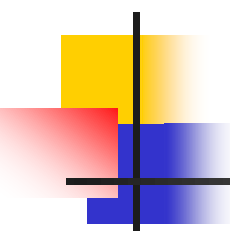
Variations of BP nets

- **Adding momentum term in Gradient Descent Learning**
 - Past increment to the weight (memory term) is used to speed up and stabilize convergence.
 - Weights update at time $t+1$ contains the momentum of the previous updates, e.g.,

$$\Delta w(t+1) = \eta \hat{o}(t) x_i + \alpha \Delta w(t)$$

Where α is Learning Rate (step size)

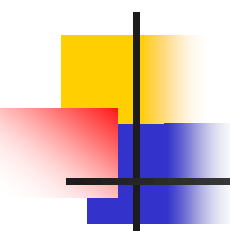
η is Momentum constant



Variations of BP nets

contd....

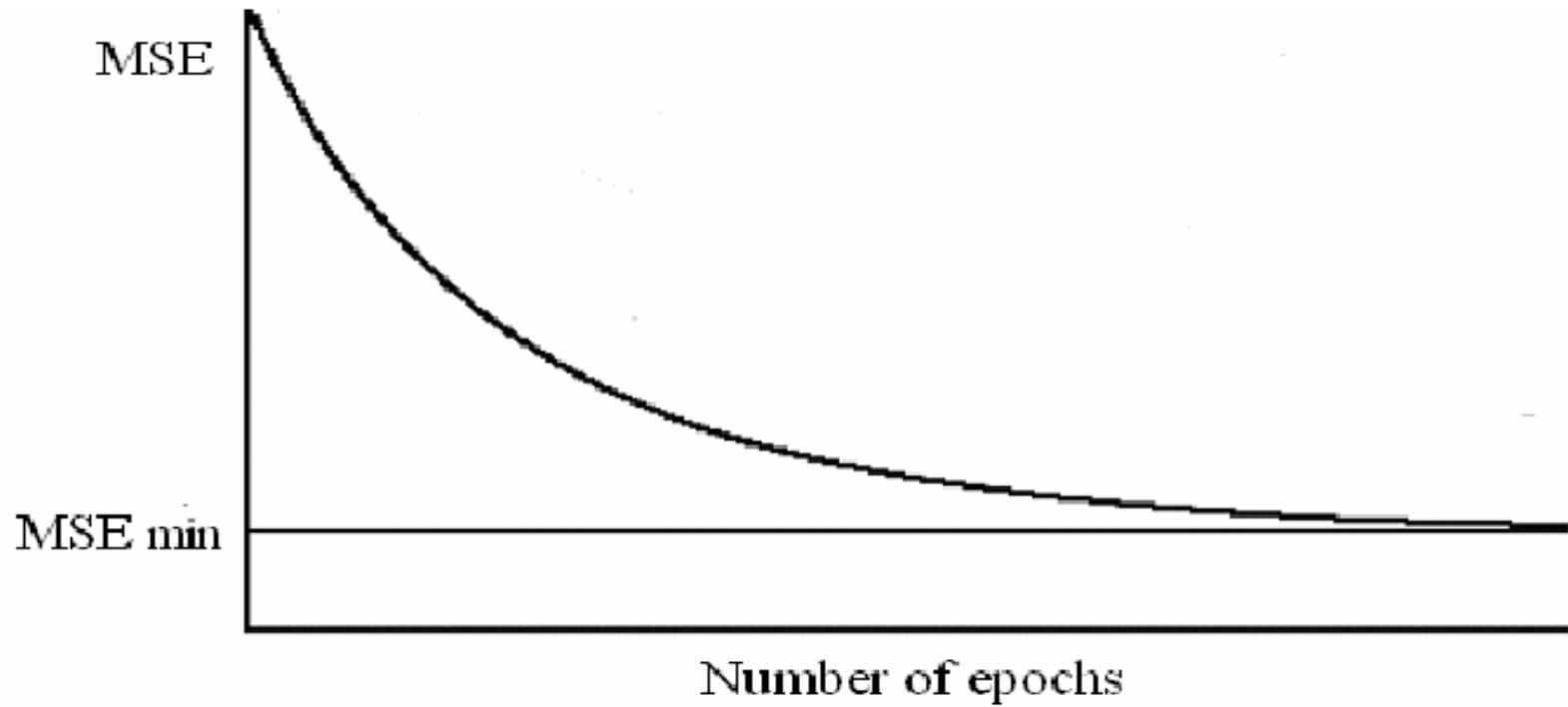
- **Batch mode** of weight updates
 - Weight update once per each epoch (cumulated over all P samples)
 - Smoothing the training sample outliers
 - Learning independent of the order of sample presentations
 - Usually slower than in sequential mode

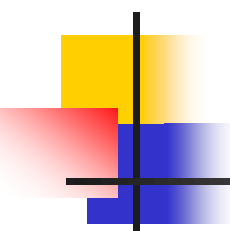


Variations in Learning Rate

- In gradient descent algorithm the step size determines how fast the algorithm moves down the gradient towards the optimal value of the parameters or weights.
- If the learning rate is too large the algorithm may diverge.
- If the learning rate is too small then the algorithm will take long time to reach the optimal position.
- Determining the best step size is highly dependent upon the data in the specific problem

Learning curve

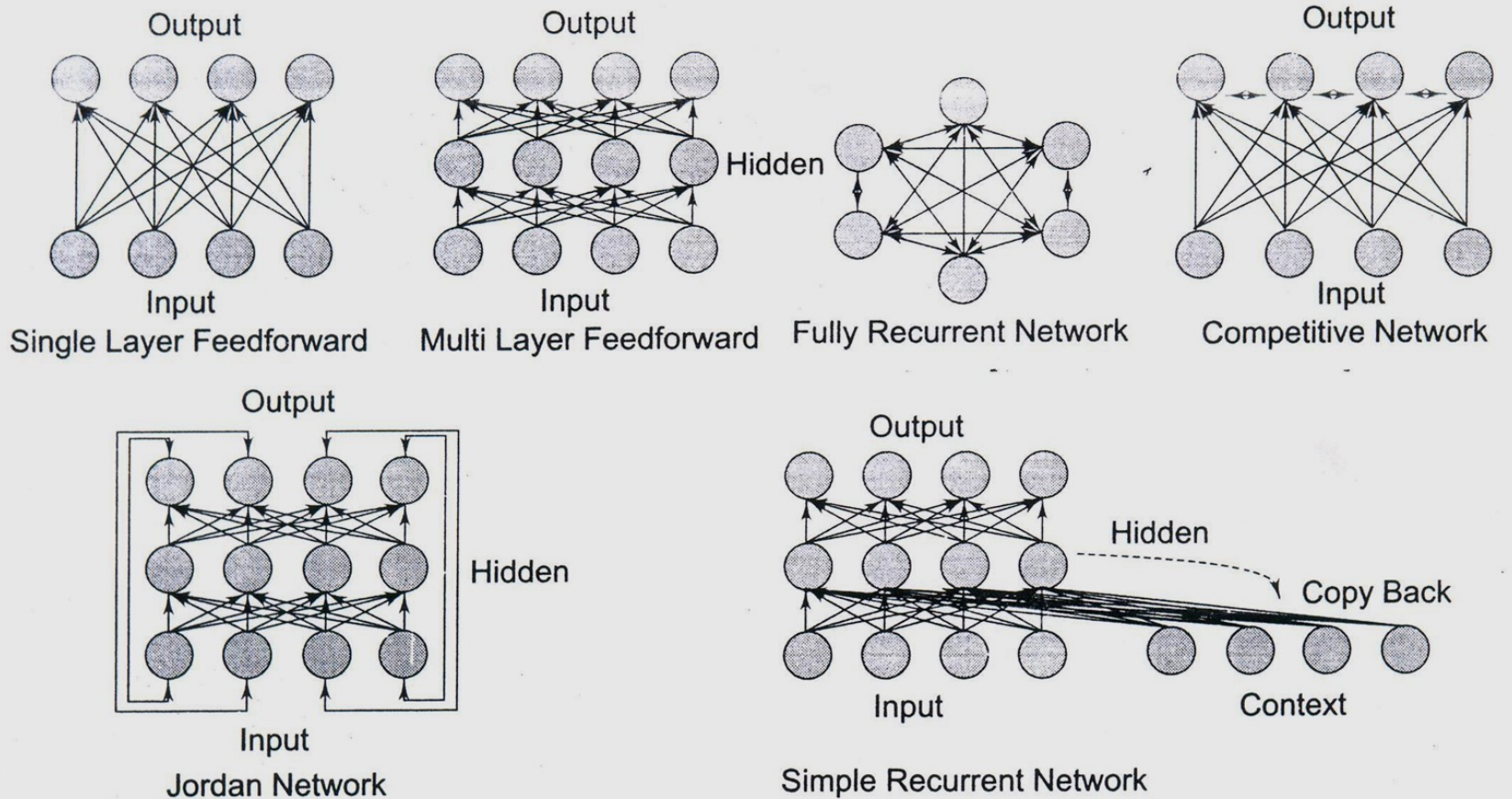


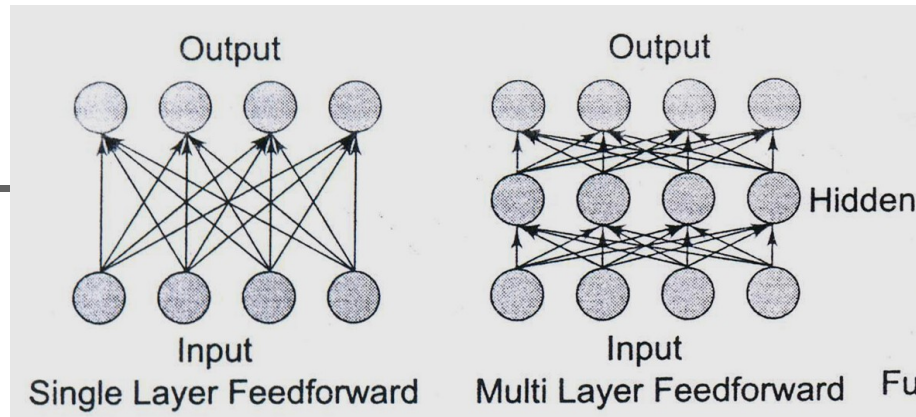


■ Error based variations on learning rate η

- Fixed rate much smaller than 1
- Start with large η , gradually decrease its value
- Start with a small η , steadily double it until MSE start to increase
- Find the maximum safe step size at each stage of learning (to avoid overshoot the minimum MSE when increasing η)

Some Neural Network structures



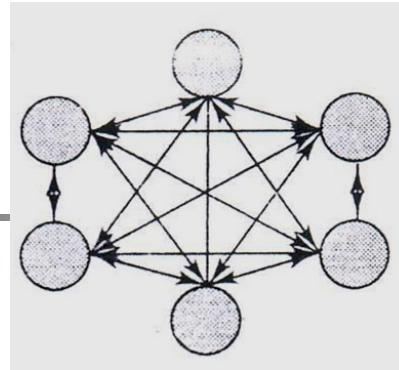
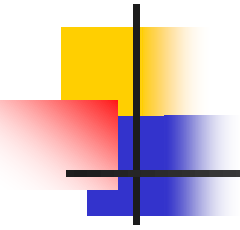


Single Layer Feedforward

- Feedforward
- One layer of Weight
- Can solve simple problems

Multi-Layer Feedforward

- Feedforward
- More than one layer of Weight (Contains hidden layer of nodes)
- Can solve more complex problems

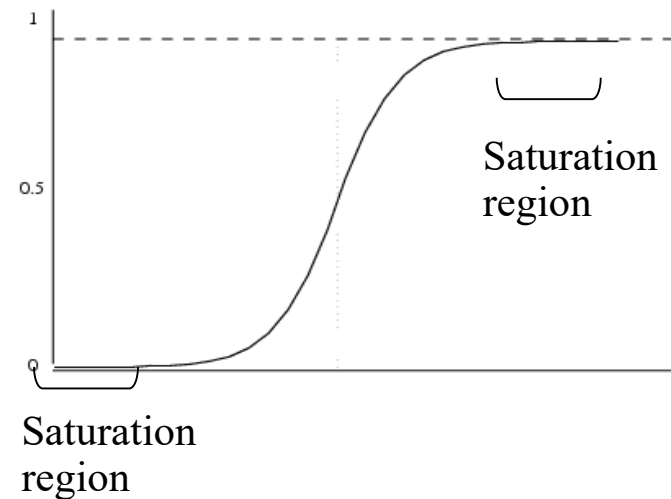


Competitive Network

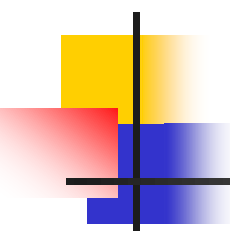
**For unipolar continuous activation function
show that $S'(x) = S(x)(1-S(x))$**

■ Differentiable:

$$\begin{aligned} S(x) &= \frac{1}{1+e^{-x}} \\ S'(x) &= -\frac{1}{(1+e^{-x})^2} \cdot (1+e^{-x})' \\ &= -\frac{1}{(1+e^{-x})^2} \cdot (-e^{-x}) \\ &= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\ &= S(x)(1-S(x)) \end{aligned}$$



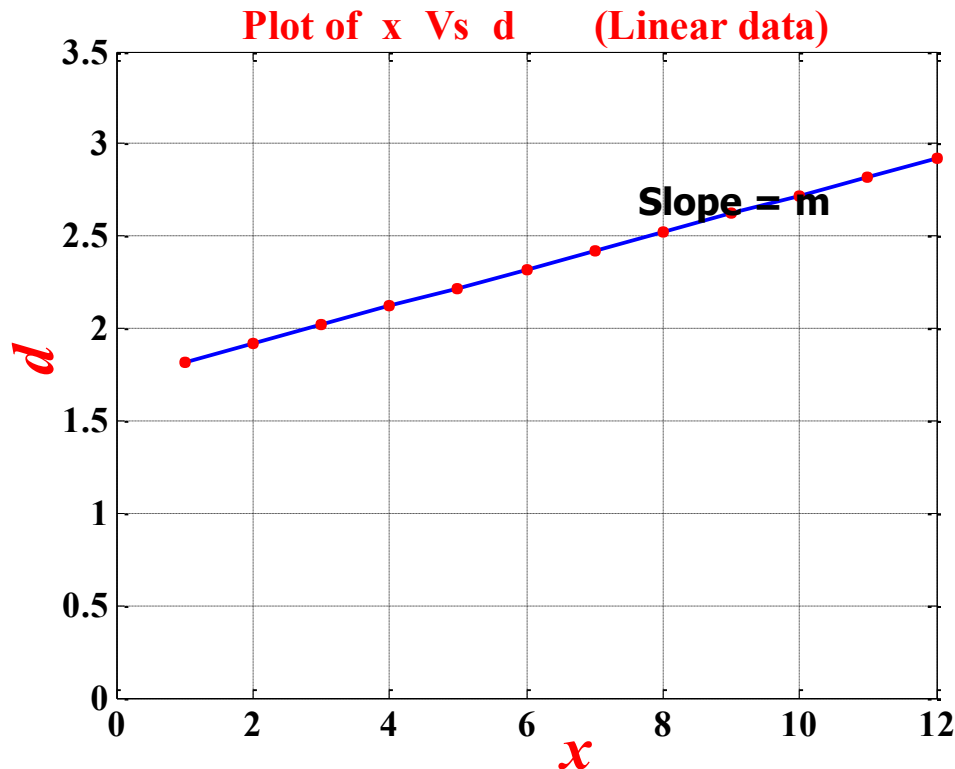
**Similarly bipolar continuous activation function
show that $S'(x) = 1/2(1-S^2(x))$**



Neural Network Problems

- Many Parameters to be set for optimal design
- Data set required for training
- Long training time in many cases e.g rainfall prediction

Applicability of Neural Networks WHERE ?



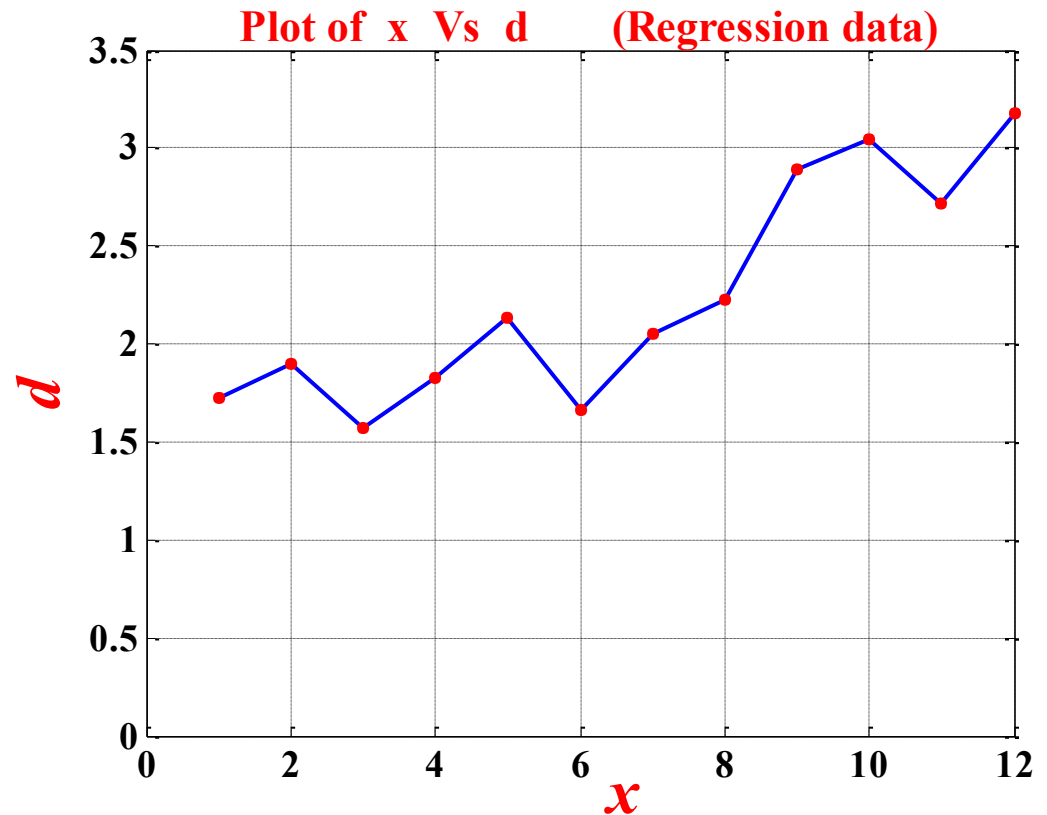
Input / output equation

$$y = mx + c$$

- Linear problem
- Solvable by Conventional Mathematics
- Need not use Neural Network

Regression (Nonlinear Problem)

x	d
1	1.72
2	1.90
3	1.57
4	1.83
5	2.13
6	1.66
7	2.05
8	2.23
9	2.89
10	3.04
11	2.72
12	3.18



Breadboard regress